

Nery Javier de la Cruz Huinil

Carné: 261233

Ejercicio #3 - D — Sistema de Control de Órdenes de Servicio

1. ¿Cuáles son las clases necesarias?

Necesito tres clases. OrdenServicio guarda los datos de una sola orden. TallerServicio tiene la lista de órdenes y todo lo que se hace con ellas: registrar, buscar, modificar, cancelar y hacer los cálculos. Main es la raíz del programa

#	Tipo de dato	Nombre	Visibilidad	Motivo	Clase
1	int	numeroOrden	private	Almacena el número consecutivo que identifica de forma única a la orden y permite ubicarla dentro del sistema.	OrdenServicio
2	String	propietario	private	Almacena el nombre completo del propietario del vehículo, dato necesario para emitir y entregar la orden.	OrdenServicio
3	String	placa	private	Almacena la placa del vehículo, con la que se pueden consultar todas las órdenes asociadas a un mismo carro.	OrdenServicio
4	String	descripcion	private	Almacena el detalle del	OrdenServicio

				trabajo solicitado, que puede actualizarse si el diagnóstico cambia.	
5	double	costoEstimado	private	Almacena el costo estimado del servicio, base para los cálculos de total, promedio y orden más alta.	OrdenServicio
6	List<OrdenServicio>	ordenes	private	Almacena el conjunto de órdenes vigentes del taller y permite que crezca o disminuya según la operación diaria.	TallerServicio
7	Scanner	entrada	private static	Recibe los datos que el usuario ingresa por teclado durante la ejecución del menú.	Main
8	TallerServicio	taller	private static	Mantiene la referencia al administrador de órdenes, para que el menú pueda ejecutar cada operación solicitada.	Main

Métodos: retorno, nombre, visibilidad y justificación

En la siguiente tabla pongo los métodos de cada clase. Los constructores los dejo con retorno «—» porque no devuelven nada.

#	Retorno	Nombre	Visibilidad	Motivo	Clase
1	—	OrdenServicio(int numeroOrden, String propietario, String placa, String descripcion, double costoEstimado)	public	Crea una orden ya completa con sus cinco datos, evitando que existan registros incompletos en el sistema.	OrdenServicio
2	int	getNumeroOrden()	public	Entrega el número de la orden cuando se necesita identificarla o verificar que no esté repetida.	OrdenServicio
3	String	getPropietario()	public	Entrega el nombre del propietario para mostrarlo en los listados y consultas.	OrdenServicio
4	String	getPlaca()	public	Entrega la placa del vehículo, dato que se compara en la consulta por placa.	OrdenServicio
5	String	getDescripcion()	public	Entrega el detalle del trabajo solicitado para mostrarlo al usuario.	OrdenServicio
6	double	getCostoEstimado()	public	Entrega el costo de la orden, valor que se utiliza en todos los cálculos del sistema.	OrdenServicio

7	void	setDescripcion(String descripcion)	public	Actualiza el detalle del trabajo cuando el diagnóstico del vehículo cambia.	OrdenServicio
8	void	setCostoEstimado(double costoEstimado)	public	Actualiza el costo estimado cuando se ajusta el precio del servicio.	OrdenServicio
9	String	toString()	public	Presenta los datos de la orden en un formato ordenado y legible para el usuario.	OrdenServicio
10	—	TallerServicio()	public	Prepara el sistema dejando la colección de órdenes lista para recibir el primer registro.	TallerServicio
11	void	registrarOrden(OrdenServicio orden)	public	Incorpora una orden nueva al sistema una vez que sus datos fueron validados.	TallerServicio
12	OrdenServicio	buscarPorNumero(int numeroOrden)	public	Localiza una orden a partir de su número, paso previo a modificarla o cancelarla.	TallerServicio
13	boolean	existeNumero(int numeroOrden)	private	Verifica si un número de orden ya fue asignado, para impedir registros duplicados.	TallerServicio

1 4	void	modificarOrden(int numeroOrden, String nuevaDescripcion, double nuevoCosto)	public	Actualiza el detalle del trabajo y el costo de una orden ya registrada.	TallerServicio
1 5	void	cancelarOrden(int numeroOrden)	public	Retira del sistema una orden que fue cancelada por el cliente o el taller.	TallerServicio
1 6	List<OrdenServicio >	consultarPorPlaca(String placa)	public	Reúne todas las órdenes de un mismo vehículo, ya que una placa puede tener varios servicios.	TallerServicio
1 7	List<OrdenServicio >	listarOrdenes()	public	Entrega el listado completo de órdenes para mostrarlo al usuario.	TallerServicio
1 8	double	calcularTotal()	public	Suma el costo de todas las órdenes para conocer el monto acumulado del taller.	TallerServicio
1 9	double	calcularPromedio()	public	Calcula el costo promedio de los servicios registrados.	TallerServicio
2 0	OrdenServicio	obtenerMayorCosto()	public	Identifica la orden con el costo más alto registrado en el sistema.	TallerServicio

2 1	int	cantidadOrdenes()	public	Informa cuántas órdenes hay registradas actualmente .	TallerServicio
2 2	void	main(String[] args)	public static	Inicia la ejecución del programa y mantiene el menú activo hasta que el usuario decide salir.	Main
2 3	void	mostrarMenu()	private static	Presenta al usuario las diez opciones disponibles del sistema.	Main
2 4	void	opcionRegistrar()	private static	Solicita los datos de la nueva orden al usuario y gestiona su registro.	Main
2 5	double	leerCosto()	private static	Obtiene el costo ingresado por el usuario y advierte cuando el valor no es numérico.	Main

4. ¿Qué tipo de objetos almacenará la colección?

La lista guarda objetos de tipo OrdenServicio. Cada objeto es una orden con sus cinco datos, así que la lista viene siendo todas las órdenes que tiene el taller en ese momento.

5. ¿Por qué se utiliza List y ArrayList en lugar de un arreglo?

Uso List y ArrayList porque no sé cuántas órdenes va a registrar el taller. Con un arreglo tendría que poner un tamaño fijo desde el inicio y si se llena ya no puedo agregar más. El ArrayList crece solo cuando agrego una orden y se reduce cuando cancelo una. También ya trae listos los métodos add(), remove(), get() y size(), así que no tengo que programarlos yo.

8. ¿Qué constructores se necesitan y con qué valores iniciales?

En OrdenServicio uso un constructor que recibe los cinco datos, para que no se pueda crear una orden a medias. En TallerServicio uso un constructor sin parámetros que crea la lista vacía, o sea que empieza con cero órdenes. Main no lleva constructor porque solo tiene métodos estáticos.

9. ¿Cómo se identifica cada orden dentro de la colección?

Cada orden se identifica por su numeroOrden, que no se puede repetir. Antes de registrar reviso con existeNumero() si ese número ya está usado, y cuando quiero modificar o cancelar busco la orden con buscarPorNumero().

10. ¿Cómo se agregan y eliminan elementos de la colección?

Para agregar uso add() dentro de registrarOrden(), pero primero valido que los datos estén bien. Para eliminar uso remove() dentro de cancelarOrden(), después de buscar la orden y comprobar que sí existe. Así la lista solo se modifica desde TallerServicio y no desde el menú.

11. ¿Cómo se realiza la búsqueda de órdenes por placa?

Recorro toda la lista y voy comparando la placa de cada orden con la que busca el usuario, usando equalsIgnoreCase() para que no importen las mayúsculas. Cada orden que coincida la guardo en otra lista y al final devuelvo esa lista, porque un mismo carro puede tener varias órdenes. Si la lista sale vacía significa que esa placa no tiene órdenes.

12. ¿Cómo se recorre la colección para calcular el total y el promedio?

Uso un ciclo for-each que va sumando el costo de cada orden en una variable. Esa suma es el total. Para el promedio divido el total entre la cantidad de órdenes, pero antes reviso que la lista no esté vacía, porque si no estaría dividiendo entre cero y el programa daría error.

13. ¿Cómo se determina la orden de mayor costo?

Guardo la primera orden como la mayor y luego recorro las demás comparando costos. Cada vez que encuentro una que cuesta más, esa se vuelve la mayor. Cuando termino de recorrer, la que quedó guardada es la orden más cara. Si la lista está vacía devuelvo null y el menú avisa que no hay órdenes.

14. ¿Qué excepciones pueden presentarse durante la ejecución?

La más común es cuando el usuario escribe letras donde va un número, ahí salta InputMismatchException o NumberFormatException. También puede pasar que quiera modificar o cancelar una orden que no existe. Y si uso el resultado de una búsqueda sin revisar que no venga null, sale NullPointerException. En el promedio puede salir error si la lista está vacía y divido entre cero.

15. ¿Qué operaciones deben protegerse con manejo de excepciones?

Protejo todo lo que el usuario escribe por teclado y que tenga que ser número: la opción del menú, el número de orden y el costo. También protejo modificar y cancelar, porque ahí la orden puede no existir. Y protejo el promedio y la orden mayor por si la lista está vacía. En todos los casos capturo el error, le muestro un mensaje al usuario y el programa regresa al menú sin cerrarse.

16. ¿Para qué se utiliza el bloque finally?

El finally siempre se ejecuta, no importa si hubo error o no. Lo uso cuando leo el costo, para limpiar lo que quedó en el Scanner y mostrar un mensaje de que la

operación terminó. Así me aseguro de que el programa siempre vuelva al menú y no se quede trabado.